

Фаза 1: Тотальный аудит и зачистка (С чистого листа)

Прежде чем строить новую систему, нужно уничтожить «хвосты» от предыдущих 101 попытки.

1. Аудит Google Cloud Platform (выполнять в Google Cloud Cloud Shell или локально через gcloud)

Выполните эти команды для понимания текущей ситуации:

Bash

```
# Просмотр активного аккаунта и проекта  
gcloud config list
```

```
# Проверка всех проектов (ищем старые и тестовые, чтобы удалить)  
gcloud projects list
```

```
# Проверка активных сервисных аккаунтов (Service Accounts) в текущем проекте  
gcloud iam service-accounts list
```

```
# Проверка включенных API (нас интересует Vertex AI)  
gcloud services list --enabled
```

Решение: Создайте абсолютно **новый проект** в GCP для этой задачи. Это изолирует биллинг и исключит конфликты старых IAM-политик.

2. Аудит локального ПК (PowerShell от имени Администратора)

Проверяем запущенные процессы, порты и старые конфигурации, которые могут конфликтовать:

PowerShell

```
# Проверка портов, которые могут быть заняты старыми агентами (обычно 3000, 4000, 8080)  
netstat -ano | findstr "3000 4000 8080"
```

```
# Удаление старых авторизаций Google Cloud на ПК (чтобы избежать конфликтов токенов)
Remove-Item -Path "$env:USERPROFILE\.config\gcloud" -Recurse -Force -ErrorAction SilentlyContinue
```

```
# Проверка наличия и статуса Cloudflare Tunnel (cloudflared)
Get-Process cloudflared -ErrorAction SilentlyContinue
```

Фаза 2: Настройка GCP и авторизации (Application Default Credentials)

1. **Создание сервисного аккаунта (SA):** В новом проекте GCP создайте SA с единственной ролью — Vertex AI User. Не давайте ему права редактора или владельца.
2. **Получение ключей:** Создайте JSON-ключ для этого SA и скачайте его на ПК (например, в C:\Keys\vertex-sa.json).
3. **Настройка локальной среды:** В PowerShell привяжите этот ключ к системе, чтобы все локальные процессы автоматически авторизовались в GCP:

PowerShell

```
[System.Environment]::SetEnvironmentVariable('GOOGLE_APPLICATION_CREDENTIALS',
'C:\Keys\vertex-sa.json', 'User')
```

Фаза 3: Развертывание шлюза LiteLLM (Ключ к бюджету и фоллбэку)

Именно LiteLLM решит задачу переключения моделей и ограничения в \$199. OpenClaw не умеет сам считать деньги и переключать модели при ошибках биллинга.

1. Установите LiteLLM на ПК (требуется Python): `pip install litellm[proxy]`
2. Создайте файл `config.yaml`. Здесь мы настраиваем Vertex AI как провайдера для всех трех моделей и задаем логику fallback:

YAML

```
model_list:
- model_name: sonnet-4.5
```

litellm_params:

model: vertex_ai/claude-3-5-sonnet-v2@20241022 # Укажите актуальный эндпоинт 4.5

vertex_project: "ВАШ_НОВЫЙ_PROJECT_ID"

vertex_location: "us-east5"

- model_name: llama-3.3

litellm_params:

model: vertex_ai/llama-3.3-70b-instruct

vertex_project: "ВАШ_НОВЫЙ_PROJECT_ID"

vertex_location: "us-central1"

- model_name: mixtral

litellm_params:

model: vertex_ai/mixtral-8x7b-instruct

vertex_project: "ВАШ_НОВЫЙ_PROJECT_ID"

vertex_location: "us-central1"

router_settings:

fallbacks: [{"sonnet-4.5": ["llama-3.3", "mixtral"]}]

litellm_settings:

max_budget: 199.00 # Жесткое ограничение бюджета в долларах

budget_duration: "3mo" # Бюджет растягивается до конца мая

3. Запустите шлюз: `litellm --config config.yaml --port 4000`.

Теперь у вас локально работает API, совместимое с OpenAI, но деньги списываются с GCP кредитов. Как только затраты превысят \$199, или Vertex AI выдаст ошибку квоты, шлюз моментально и бесшовно перекинет запрос OpenClaw на Llama 3.3.

Фаза 4: Установка и привязка OpenClaw

OpenClaw теперь не должен ничего знать про Google Cloud. Он должен думать, что работает с локальным сервером.

1. В конфигурации OpenClaw (или через UI, если запускаете `openclaw dashboard`) выберите провайдера **OpenAI-Compatible** (или LiteLLM, если доступно).
2. Базовый URL (Base URL): `http://127.0.0.1:4000`
3. Ключ API: любой случайный текст (LiteLLM по умолчанию пустит локальный запрос, если не настроены отдельные ключи).
4. Модель: `sonnet-4.5`.

Фаза 5: Интеграция с Telegram и iPhone (Cloudflare Tunnels)

Чтобы безопасно пробросить доступ к OpenClaw на iPhone и для Telegram-бота без открытия портов на роутере:

1. Загрузите и установите `cloudflared` для Windows.

2. Авторизуйтесь: `cloudflared tunnel login`
3. Создайте туннель: `cloudflared tunnel create openclaw-tunnel`
4. Сконфигурируйте маршрутизацию. Создайте файл конфигурации `config.yml`:

YAML

`tunnel: <ID_вашего_туннеля>`

`credentials-file: C:\Users\БАШ_USER\.cloudflared\<ID>.json`

`ingress:`

`- hostname: agent.yourdomain.com # Для доступа с iPhone (Web UI)`

`service: http://localhost:18789 # Порт UI OpenClaw`

`- hostname: webhook.yourdomain.com # Для Telegram webhook`

`service: http://localhost:3000 # Порт, на котором слушает интеграция TG`

`- service: http_status:404`

5. Запустите туннель: `cloudflared tunnel run`.
6. **Telegram Bot:** В настройках интеграции каналов OpenClaw введите токен вашего бота (от BotFather) и укажите публичный URL вебхука `https://webhook.yourdomain.com/telegram`.

Контроль GCP бюджета (Резервный слой):

Хотя LiteLLM контролирует расход на уровне API, обязательно зайдите в Google Cloud Billing -> Budgets & alerts. Установите там бюджет \$190 на проект с триггерами на 50%, 90% и 100%. Это будет вашей абсолютной гарантией того, что кредитные средства не улетят в минус до 29 мая 2026 года.